BFS AND DFS

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    traversal = []

    while queue:
        node = queue.popleft()
        traversal.append(node)

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return traversal



def dfs(graph, start):
    visited = set()
    traversal = []

    def dfs_util(node):
        visited.add(node)
        traversal.append(node)

        for neighbor in graph[node]:
```

```
            if neighbor not in visited:
                dfs_util(neighbor)

    dfs_util(start)
    return traversal


# Test Case 1: Connected graph
graph1 = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B'],
    'F': ['C']
}

print("Test Case 1: Connected graph")
print("BFS:", bfs(graph1, 'A'))
print("DFS:", dfs(graph1, 'A'))


# Test Case 2: Multiple branches
graph2 = {
    'A': ['B', 'C'],
    'B': ['E', 'F'],
    'C': ['D', 'G'],
    'D': ['H'],
    'E': [], 'F': [], 'G': [], 'H': [], 'I': []
```

```
}

print("\nTest Case 2: Multiple branches")
print("BFS:", bfs(graph2, 'A'))
print("DFS:", dfs(graph2, 'A'))


# Test Case 3: Cyclic graph
graph3 = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C']
}

print("\nTest Case 3: Cyclic graph")
print("BFS:", bfs(graph3, 'A'))
print("DFS:", dfs(graph3, 'A'))


# Test Case 4: Single-node graph
graph4 = {'A': []}

print("\nTest Case 4: Single-node graph")
print("BFS:", bfs(graph4, 'A'))
print("DFS:", dfs(graph4, 'A'))


# Test Case 5: Disconnected graph
```

```
graph5 = {
    'A': ['B'], 'B': ['A'],
    'C': ['D'], 'D': ['C'], 'E': []
}

print("\nTest Case 5: Disconnected graph")
print("BFS:", bfs(graph5, 'A'))
print("DFS:", dfs(graph5, 'A'))
```

SVM

```
from sklearn.datasets import load_iris
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np
import matplotlib.pyplot as plt

iris = load_iris()
X, y = iris.data, iris.target

# TC1: Two classes
X1, y1 = X[y != 2], y[y != 2]
model1 = SVC(kernel="linear")
model1.fit(X1, y1)
print("TC1 Accuracy:", accuracy_score(y1, model1.predict(X1)))
```

```
# TC2: Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X1, y1, test_size=0.2, random_state=42
)
model2 = SVC(kernel="linear")
model2.fit(X_train, y_train)
print("TC2 Accuracy:", accuracy_score(y_test,
model2.predict(X_test)))

# TC3: Predict one sample
model3 = SVC(kernel="linear")
model3.fit(X1, y1)
point = X1[0].reshape(1, -1)
print("TC3 Predicted class:", model3.predict(point)[0])
print("TC3 Decision value:",
model3.decision_function(point)[0])

# TC4: Add noise
rng = np.random.default_rng(42)
Xu = X1 + rng.normal(0, 0.8, X1.shape)
model4 = SVC(kernel="linear")
model4.fit(Xu, y1)
print("TC4 Accuracy:", accuracy_score(y1, model4.predict(Xu)))

# TC5: Single class
X5, y5 = X[y == 0], y[y == 0]
try:
    model5 = SVC(kernel="linear")
    model5.fit(X5, y5)
except ValueError:
```

```
    print("TC5: SVM needs at least two classes.")

# Graphical representation
Xg = X1[:, [0, 1]]
graph_model = SVC(kernel="linear")
graph_model.fit(Xg, y1)

plt.scatter(Xg[y1 == 0, 0], Xg[y1 == 0, 1], label="Class 0")
plt.scatter(Xg[y1 == 1, 0], Xg[y1 == 1, 1], label="Class 1")

xx = np.linspace(Xg[:, 0].min(), Xg[:, 0].max(), 100)
w = graph_model.coef_[0]
b = graph_model.intercept_[0]
yy = -(w[0] * xx + b) / w[1]

plt.plot(xx, yy, label="Hyperplane")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("SVM Classification")
plt.legend()
plt.grid()
plt.show()
```

A*

```
def a_star(graph, h, start, goal):
    open_list = [start]
    closed = set()
    g = {start: 0}
    parent = {start: start}

    while open_list:
        n = min(open_list, key=lambda x: g[x] + h[x])

        if n == goal:
            path = []
            current = n

            while parent[current] != current:
                path.append(current)
                current = parent[current]

            path.append(start)
            path.reverse()

            print("Path:", path)
            print("Cost:", g[goal])
            return path, g[goal]

        open_list.remove(n)
```

```
        closed.add(n)

        for m, cost in graph[n].items():
            new_cost = g[n] + cost

            if m not in g or new_cost < g[m]:
                g[m] = new_cost
                parent[m] = n

                if m in closed:
                    closed.remove(m)

                if m not in open_list:
                    open_list.append(m)

    print("Path not found")
    return None, float("inf")



# Test Case 1
print("Test Case 1")
graph = {
    'A': {'B': 1, 'C': 3, 'G': 2},
    'B': {'E': 2},
    'C': {'G': 3},
    'E': {'G': 3},
    'G': {}
}
```

```
h = {'A': 2, 'B': 3, 'C': 1, 'E': 1, 'G': 0}
a_star(graph, h, 'A', 'G')


# Test Case 2
print("\nTest Case 2")
graph = {
   'S': {'A': 2, 'B': 3},
   'A': {'G': 4},
   'B': {'G': 3},
   'G': {}
}
h = {'S': 5, 'A': 4, 'B': 2, 'G': 0}
a_star(graph, h, 'S', 'G')


# Test Case 3
print("\nTest Case 3")
graph = {'A': {'G': 2}, 'G': {}}
h = {'A': 1, 'G': 0}
a_star(graph, h, 'A', 'G')


# Test Case 4
print("\nTest Case 4")
graph = {'A': {'B': 2}, 'B': {}, 'G': {}}
h = {'A': 2, 'B': 1, 'G': 0}
a_star(graph, h, 'A', 'G')
```

```
# Test Case 5
print("\nTest Case 5")
graph = {
   'S': {'A': 1, 'B': 2},
   'A': {'G': 10},
   'B': {'G': 3},
   'G': {}
}
h = {'S': 0, 'A': 0, 'B': 100, 'G': 0}
a_star(graph, h, 'S', 'G')
```

Back Propagation Algorithm

```
 import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error


def show(title, X, y, hidden=3, lr=0.1, epochs=10000):
   print("\n", title)
   m = MLPRegressor(hidden_layer_sizes=(hidden,),
                    learning_rate_init=lr,
                    max_iter=epochs, random_state=42)
   m.fit(X, y)
   p = m.predict(X)
   print("Actual:", y)
   print("Predicted:", np.round(p, 2))
```

```
    return m, p

# TC1: XOR Dataset
X1 = np.array([[0,0], [0,1], [1,0], [1,1]])
y1 = np.array([0,100,100,0])
show("TC1: XOR DATASET", X1, y1)

# TC2: Small Neural Network
X = np.array([[2,9], [1,5], [3,6]])
y = np.array([92,86,89])
show("TC2: SMALL NETWORK", X, y, hidden=1)

# TC3: Training Error
m, p = show("TC3: TRAINING ERROR", X, y)
loss = m.loss_curve_
print("Final MSE:", round(mean_squared_error(y, p), 6))

plt.plot(loss)
plt.xlabel("Epochs")
plt.ylabel("Training Loss")
plt.title("Training Error Reduction")
plt.grid()
plt.show()

# TC4: Very High Learning Rate
print("\nTC4: VERY HIGH LEARNING RATE")
m, p = show("HIGH LEARNING RATE", X, y, lr=10)
print("Learning Rate:", 10)
```

```
print("Result: A very high learning rate can make training unstable.")


# TC5: Zero Epochs
print("\nTC5: ZERO EPOCHS")
try:
    m = MLPRegressor(max_iter=0, random_state=42)
    m.fit(X, y)
except ValueError as e:
    print("Error:", e)


print("\nAll 5 test cases completed")
```

4.implement Knowledge Representation

```
from sympy import symbols, Not, Implies, Xor, satisfiable

Rain, Hagrid, Dumbledore = symbols('Rain Hagrid Dumbledore')

def run_test(name, kb):
    print("\n" + name)
    result = satisfiable(kb)
    print("Satisfiable:", result)

    if result is False:
        print("Contradiction")
    elif Rain in result:
        print("It rained today." if result[Rain]
              else "There is no rain today.")
    else:
        print("Rain cannot be determined.")

base = Implies(Not(Rain), Hagrid) & Xor(Hagrid, Dumbledore)

# TC1
kb1 = base & Dumbledore
run_test("TEST CASE 1", kb1)

# TC2
kb2 = base & Hagrid & Not(Rain)
run_test("TEST CASE 2", kb2)
```

```
# TC3
kb3 = base & Hagrid & Dumbledore
run_test("TEST CASE 3", kb3)

# TC4
kb4 = base & Rain
run_test("TEST CASE 4", kb4)

# TC5
kb5 = base & Not(Hagrid) & Not(Dumbledore)
run_test("TEST CASE 5", kb5)

print("\nAll 5 test cases completed")
```

09 K-Means Clustering algorithm

```
from sklearn import datasets
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

X = datasets.load_iris().data
plt.figure(figsize=(12, 7))

# TC1: 2 Clusters
m1 = KMeans(n_clusters=2, random_state=0, n_init=10).fit(X)
plt.subplot(2, 2, 1)
plt.scatter(X[:, 2], X[:, 3], c=m1.labels_)
plt.title("TC1: 2 Clusters")
```

```
# TC2: 3 Clusters
m2 = KMeans(n_clusters=3, random_state=0, n_init=10).fit(X)
plt.subplot(2, 2, 2)
plt.scatter(X[:, 2], X[:, 3], c=m2.labels_)
plt.title("TC2: 3 Clusters")

# TC3: Convergence
print("TC3: Iterations:", m2.n_iter_)
print("Centroids:\n", m2.cluster_centers_)
plt.subplot(2, 2, 3)
plt.scatter(X[:, 2], X[:, 3], c=m2.labels_)
plt.title("TC3: Converged")

# TC4: K > Data Points
print("\nTC4: K > Data Points")
try:
  KMeans(n_clusters=4).fit(X[:3])
except ValueError as e:
  print("Error:", e)

# TC5: Duplicate Points
D = [[1,2], [1,2], [3,4], [3,4], [5,6]]
m5 = KMeans(n_clusters=3, random_state=0, n_init=10).fit(D)
print("\nTC5: Labels:", m5.labels_)
print("Centroids:\n", m5.cluster_centers_)

plt.subplot(2, 2, 4)
```

```
plt.scatter([i[0] for i in D], [i[1] for i in D], c=m5.labels_)
plt.title("TC5: Duplicate Points")

plt.tight_layout()
plt.show()
```

Bayesian Network.

```
P_burglary = 0.002

P_earthquake = 0.001


P_alarm = {

  (True, True): 0.94,

  (True, False): 0.95,

  (False, True): 0.31,

  (False, False): 0.001

}


P_david = {True: 0.91, False: 0.05}

P_sophia = {True: 0.75, False: 0.02}



def joint_probability(a, b, e, d, s):

  if any(type(x) != bool for x in [a, b, e, d, s]):

    return "Invalid input"


  if len(P_alarm) != 4:

    return "Missing CPT values"


  probs = [P_burglary, P_earthquake] + list(P_alarm.values()) + list(P_david.values()) +
list(P_sophia.values())


  if any(not 0 <= p <= 1 for p in probs):

    return "Invalid probability"
```

```
    pa = P_alarm[(b, e)] if a else 1 - P_alarm[(b, e)]
    pd = P_david[a] if d else 1 - P_david[a]
    ps = P_sophia[a] if s else 1 - P_sophia[a]

    return (P_burglary if b else 1 - P_burglary) * \
        (P_earthquake if e else 1 - P_earthquake) * pa * pd * ps



# TC1
print("TEST CASE 1")
print("Joint Probability =", f"{joint_probability(True,False,False,True,True):.10f}")

# TC2
print("\nTEST CASE 2")
print("Joint Probability =", f"{joint_probability(True,True,False,True,True):.10f}")

# TC3
print("\nTEST CASE 3")
print("Joint Probability =", f"{joint_probability(True,False,True,True,False):.10f}")

# TC4: Missing CPT
print("\nTEST CASE 4")
temp = P_alarm.pop((False,False))
print("Result =", joint_probability(True,False,False,True,True))
P_alarm[(False,False)] = temp

# TC5: Invalid Probability
```

```
print("\nTEST CASE 5")
temp = P_alarm[(True,True)]
P_alarm[(True,True)] = 1.5
print("Result =", joint_probability(True,True,True,True,True))
P_alarm[(True,True)] = temp

print("\nAll 5 test cases completed")
```

ID3 algorithm.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.preprocessing import LabelEncoder

def run(data, features, name):
  if not data:
    raise ValueError("Empty dataset")

  df = pd.DataFrame(data, columns=features + ["Play"])

  for col in df.columns:
    df[col] = LabelEncoder().fit_transform(df[col])

  X, y = df[features], df["Play"]
  m = DecisionTreeClassifier(criterion="entropy")
  m.fit(X, y)

  print("\n", name)
```

```
    print("Accuracy:", m.score(X, y) * 100, "%")
    print(export_text(m, feature_names=features))

# TC1: Play Tennis
play_tennis = [
    ["Sunny","Hot","High","Weak","No"],
    ["Sunny","Hot","High","Strong","No"],
    ["Overcast","Hot","High","Weak","Yes"],
    ["Rain","Mild","High","Weak","Yes"],
    ["Rain","Cool","Normal","Weak","Yes"],
    ["Rain","Cool","Normal","Strong","No"],
    ["Overcast","Cool","Normal","Strong","Yes"],
    ["Sunny","Mild","High","Weak","No"],
    ["Sunny","Cool","Normal","Weak","Yes"],
    ["Rain","Mild","Normal","Weak","Yes"],
    ["Sunny","Mild","Normal","Strong","Yes"],
    ["Overcast","Mild","High","Strong","Yes"],
    ["Overcast","Hot","Normal","Weak","Yes"],
    ["Rain","Mild","High","Strong","No"]
]

run(play_tennis, ["Weather","Temperature","Humidity","Wind"], "TC1: Play Tennis")

# TC2: Clear Class Separation
clear_data = [
    ["Red","Yes"], ["Red","Yes"], ["Red","Yes"],
```

```
    ["Blue","No"], ["Blue","No"]
]
run(clear_data, ["Color"], "TC2: Clear Separation")


# TC3: Mixed Attributes
mixed_data = [
    ["Young","High","Yes"],
    ["Young","Low","No"],
    ["Middle","High","Yes"],
    ["Middle","Low","Yes"],
    ["Old","High","Yes"],
    ["Old","Low","No"]
]
run(mixed_data, ["Age","Income"], "TC3: Mixed Attributes")


# TC4: Single Class
single_data = [
    ["Red","Yes"], ["Blue","Yes"], ["Green","Yes"]
]
run(single_data, ["Color"], "TC4: Single Class")


# TC5: Empty Dataset
print("\nTC5: EMPTY DATASET")
try:
    run([], ["Color"], "Empty Dataset")
```

```
except ValueError as e:
    print("Error:", e)


print("\nAll 5 test cases completed")
```